

BUSINESS ANALYTICS PROJECT REPORT

High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

Student Name	Balaji MJ
Register Number	TU-6253210211004
Week	Week 4
Subject	Business Analytics
Task Name	High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

Task Description

Students must build a frequent pattern mining engine from scratch. They are required to design dynamic tree structures utilizing node link pointers, recursively construct conditional FP-trees, and extract high-confidence association rules from massive transactional datasets under strict execution-time constraints.

1. Project Objective

The objective is to implement an efficient frequent pattern mining engine based on the FP-Growth approach. The system should compress transactional data into an FP-tree, avoid repeated candidate generation, recursively mine conditional pattern bases and conditional FP-trees, and generate association rules that satisfy a specified confidence threshold.

The implementation is intended to work efficiently on large transactional datasets where execution time and memory usage are important considerations.

2. Transactional Data Input

The engine accepts a collection of transactions. Each transaction contains a set or list of items purchased or observed together. Before tree construction, item frequencies are counted and items that do not satisfy the selected minimum support threshold are removed.

Example transaction format:

```
T1 = {Milk, Bread, Butter}
T2 = {Bread, Diapers, Beer, Eggs}
T3 = {Milk, Bread, Diapers, Beer}
T4 = {Bread, Milk, Diapers, Cola}
```

Input processing:

1. Read transactions.
2. Count item frequency.
3. Apply minimum support.
4. Sort surviving items by descending frequency.
5. Insert ordered transactions into the FP-tree.

The supplied task does not specify a particular dataset, support threshold, or programming language. These values should therefore be treated as configurable implementation parameters.

3. Compressed FP-Tree Structure

An FP-tree is a compact prefix-tree representation of the frequent items in the transaction database. Transactions sharing common prefixes reuse the same tree paths, reducing duplicated storage.

Each tree node can maintain an item identifier, count, parent pointer, child references, and a node-link pointer. The node-link connects nodes representing the same item so that all occurrences can be traversed efficiently.

```
class FPNode:
    item
    count
    parent
    children
    node_link
```

Header Table:

```
item -> support count + first node in node-link chain
```

4. Node-Link Pointers and Header Table

The header table stores each frequent item together with its support information and a pointer to the first corresponding node in the FP-tree. Additional nodes containing the same item are connected through node-link pointers.

This structure allows the mining procedure to locate all nodes for an item without scanning the complete tree repeatedly. It is especially useful when constructing conditional pattern bases.

Component	Role
FP-tree root	Starting point of the compressed tree
FP-node	Stores item and occurrence count
Parent pointer	Connects a node to its parent
Children	Stores outgoing tree branches
Node link	Connects nodes containing the same item
Header table	Provides direct access to item node chains

5. Conditional Pattern Base Construction

For a selected item, the mining procedure follows its node-link chain. For each occurrence, the parent pointers are followed toward the root to recover the prefix path that leads to that item. These prefix paths form the conditional pattern base.

```
For each item X:
  traverse X's node-link chain
  for each node:
    follow parent pointers to root
    collect the prefix path
    attach the node count to that path
```

```
Result:
Conditional pattern base(X)
```

The conditional pattern base is then filtered using the minimum support requirement before a conditional FP-tree is constructed.

6. Recursive Conditional FP-Trees

A conditional FP-tree is built from the conditional pattern base of an item. The same process is applied recursively to discover progressively longer frequent itemsets.

```
FP_Growth(tree, suffix):
    if tree contains a single path:
        generate combinations from the path
    else:
        for each item in header table:
            new_suffix = item + suffix
            output new_suffix as a frequent pattern
            base = conditional_pattern_base(item)
            conditional_tree = build_FP_tree(base)
            if conditional_tree is not empty:
                FP_Growth(conditional_tree, new_suffix)
```

The recursive structure enables pattern extraction without explicitly generating the large candidate sets associated with classical Apriori-style candidate generation.

7. Association Rule Generation

After frequent itemsets are extracted, association rules can be generated by dividing an itemset into an antecedent and consequent. Confidence measures how often the consequent occurs when the antecedent occurs.

```
confidence(A -> B)
  = support(A union B) / support(A)
```

Example:

```
If support({Bread, Milk}) = 40%
and support({Bread}) = 50%,
```

```
confidence(Bread -> Milk)
  = 0.40 / 0.50
  = 80%
```

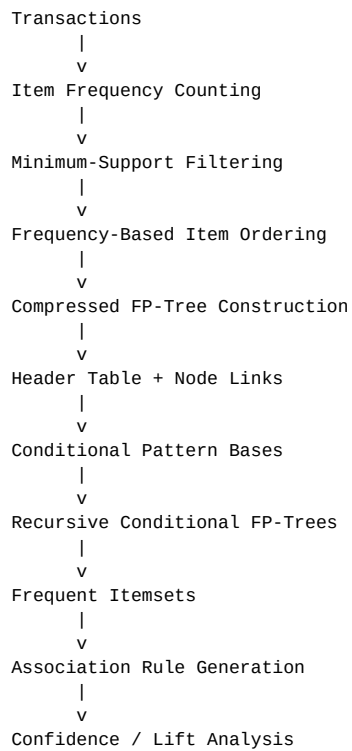
Rules can be retained when their confidence is greater than or equal to the configured minimum confidence threshold. Additional measures such as lift may also be calculated when required by the implementation.

8. High-Performance Design

The implementation should minimize repeated database or dataset scans and make efficient use of the compressed tree representation. Important performance considerations include compact node storage, direct header-table access, node-link traversal, transaction sorting, and recursive conditional-tree mining.

Optimization Area	Implementation Focus
Compression	Reuse common transaction prefixes
Lookup	Use header table for direct item access
Traversal	Use node-link chains instead of full-tree scans
Filtering	Remove infrequent items early
Memory	Avoid unnecessary duplicate transaction structures
Recursion	Mine only non-empty conditional trees
Execution	Measure runtime on large transaction sets

9. End-to-End Workflow



The complete pipeline transforms raw transactional records into frequent patterns and high-confidence association rules while keeping the mining process focused on the compressed FP-tree representation.

10. Technologies, Testing and Deliverables

A suitable implementation can be developed in Python or another language that supports dynamic data structures and recursion. The exact technology stack is not specified in the task description, so the project should use the environment prescribed by the course or instructor.

Testing: Validate support counts against small manually checked datasets, verify node-link chains, test conditional-tree construction, confirm association-rule confidence calculations, and measure execution time and memory behavior as dataset size increases.

Expected Deliverables:

1. Source code implementing the FP-tree and FP-Growth engine from scratch.
2. Transaction preprocessing and support filtering module.
3. Header-table and node-link implementation.
4. Conditional pattern-base and conditional FP-tree generation.
5. Frequent itemset output.
6. High-confidence association-rule output.
7. Runtime and performance measurements.
8. Project report documenting design and results.

Conclusion

This Week-4 Business Analytics task focuses on implementing high-performance market basket analysis using compressed FP-trees. The project combines dynamic tree structures, node-link traversal, recursive conditional FP-tree construction, frequent pattern mining, and association-rule generation for large transactional datasets.